
Install documentation of ELI Annotation Tool

Release 1.0.0

Publications Office of the European Union

2017-12-08

CONTENTS

1	Overview	3
2	Install	5
2.1	Hardware prerequisites	5
2.2	Setup	5
2.3	Install	5
2.4	Web server configuration	6
2.5	Configuration	6
3	Command-line operations	7
3.1	Configuration	7
3.2	Users management	9
3.3	Localization	11
3.4	Starting the application	13
3.5	Running the application as a daemon	14
4	License	15

The ELI Annotation Tool has been developed in the framework of the [European Legislation Identifier](http://www.eli.fr/) (<http://www.eli.fr/>) initiative which is undertaken with the support of [ISA²](https://ec.europa.eu/isa2) (<https://ec.europa.eu/isa2>).

[ISA²](https://ec.europa.eu/isa2) (<https://ec.europa.eu/isa2>) is a EUR 131 million programme of the European Commission which develops digital solutions that enable interoperable cross-border and cross-sector public services for the benefit of public administrations, businesses and citizens across the EU.

[ISA²](https://ec.europa.eu/isa2) (<https://ec.europa.eu/isa2>) supports a large range of [actions](https://ec.europa.eu/isa2/actions_en) (https://ec.europa.eu/isa2/actions_en) and [solutions](https://ec.europa.eu/isa2/solutions_en) (https://ec.europa.eu/isa2/solutions_en). The [ISA²](https://ec.europa.eu/isa2) (<https://ec.europa.eu/isa2>) solutions can be used free of charge and are open source when related to IT.

[ISA²](https://ec.europa.eu/isa2) (<https://ec.europa.eu/isa2>) - IT solutions for less bureaucracy: You click, we link.

OVERVIEW

The ELI annotation tool is a tool for building notices describing legal resources using the [European Legislation Identifier](http://www.eli.fr/) (<http://www.eli.fr/>) (ELI) standard.

The notices describe several standard legal entities (*legal resources*, *legal expressions* and *formats*) using standard properties. The values of some of these properties are constrained using controlled vocabularies described with the [SKOS standard](https://www.w3.org/2004/02/skos/) (<https://www.w3.org/2004/02/skos/>).

The notices can be created for three different types of resources: *acts*, *official journals* and *consolidations*. For each of these types, the ELI annotation tool allows to configure the notices form by choosing which properties will be active for the legal entities and by choosing the controlled vocabularies of these properties (when applicable).

The ELI annotation tool is a Web application that must be installed on a Web server. Command-line operations executed on the server allow to configure the ELI annotation tool, localize it and manage its users. Once this server is configured, the ELI annotation tool is accessible simply with a recent (as of end of 2017) Web browser.

The ELI annotation tool is not meant to publish, manage or archive notices and vocabularies. It is meant to be simple to set up (no database, no connection to a user directory, no connection to external data sources, etc.) and to allow users to efficiently produce the notices that will be published on the Web using other tools and other servers.

INSTALL

The ELI Annotation tool is expected to be installed onto a Debian Stretch (Debian 9) system thanks to a deb package.

Hardware prerequisites

The ELI Annotation tool is a Web application and should be installed on a computer that meets the state-of-the-art requirements of a standard little Web server. Currently (as of end 2017), this would be:

- RAM: 4 Go
- Disk: 32 Go
- CPU: Intel i7 or equivalent

The CPU could be a less powerful one but then, when uploading big vocabularies, the processing might take too long.

Setup

Add the stretch-backports repository to the package sources, by adding to the `/etc/apt/source.list.d` directory a file named `stretch-backports.list` that contains the line:

```
deb http://httpredir.debian.org/debian stretch-backports main
```

This will allow the retrieval of the needed dependencies (listed in the sources in the `debian/control` file).

Install

Install the ELI Annotation Tool package using the `dpkg` tool:

```
dpkg -i eli-annotation-tool.deb
```

Then make sure all the dependency packages are installed with:

```
apt-get -f install
```

Web server configuration

The ELI annotation tool is a Web application that listens on a network port (see *Starting the application* (page 13) section). If needed, make sure your firewall is not blocking the HTTP traffic to this port on your server.

Please note that the ELI annotation tool is a Web application that expects its URLs to be absolute (main page at /). If you use a proxy server, make sure it is correctly configured for the URL routing process to work.

Configuration

Please note that **it is necessary to configure the tool before doing any other operation.**

COMMAND-LINE OPERATIONS

On the server where it is installed, the command-line of ELI Annotation tool can be invoked with:

```
python3 -m eli_annotation
```

The command-line can be used to:

- configure the tool
- manage users
- localize the GUI
- start the application Web server

Please note that **it is necessary to configure the tool before doing any other operation.**

Configuration

The following command starts the configuration:

```
python3 -m eli_annotation configure
```

Firstly, the tool lists the various languages corresponding to the existing localization files (cf. [Localization](#) (page 11)). It asks for the language that will be used for the user interface (localization of the various UI messages).

The ELI annotation tool produces HTML pages describing the uploaded SKOS vocabularies (cf. user manual). These HTML pages are generated in one or several languages. The next question of the configuration process consists in choosing the languages for these vocabulary HTML pages. The languages are chosen amongst the previously listed languages. Several languages can be specified (separated by spaces).

The notices produced by the ELI annotation tool contain properties describing the user that created them. Each user has a distinct URI built from a common URI prefix followed by the user's identifier. The next question of the configuration concerns the common URI prefix used for the users' URI. Please be aware that the user's identifier is directly concatenated to the prefix. Therefore, the prefix might end with a character such as / or #.

The ELI annotation tool produces HTML pages describing the defined legal notices (cf. user manual). These pages contain an embedded semantic description of the legal entities (in RDFa) and a visual rendering of the legal entities (into HTML tables that can be seen in a browser). It is possible to choose to generate:

- HTML pages that only contain the embedded semantic description of the legal entities (such pages are totally blank when displayed in a browser),
- HTML pages that contain the embedded semantic description of the legal entities and the visual rendering of these entities (such pages display several properties tables in a browser)

The last question of the configuration allows choosing between these two alternatives. Most of the times, the second one will be chosen (HTML pages with visual content).

When the configuration is done for the first time, various standard resources will be deployed: standard localization files (notably the English one), other standard localized elements such as the about text and the help on the Excel vocabulary format, and finally the standard vocabularies (EU languages, IANA media formats, ELI vocabularies).

The following screenshot shows a complete configuration.

```
python3 -m eli_annotation configure

*****
** ELI Annotation Tool - Configuration **
*****

* Deploying standard localization files...

* Languages

Available languages are: en fr

Choose the language of the user interface (one code)
? fr

Choose the languages for the vocabulary rendering in HTML (one or several
codes separated by spaces)
? en fr

* User URIs

Choose the URI prefix for the users (will be added before the user ID to
build an URI; this URI indicates who created a notice)
? http://data.legi-org.eu/users/

* HTML rendering

The HTML rendering of the vocabularies and the notices can contain:
[1] Only metadata inside the HTML header for describing the objets in
RDFa and schema.org standards
[2] metadata inside the HTML header AND actual content in the HTML body
for displaying the objects in human-readable text

If you don't know what to choose, choose 2.

Enter your choice (1 or 2)
? 2

* Deploying standard about files...

* Deploying standard excel_vocab_doc files...
```

```
* Deploying standard vocabularies...
```

Users management

Three commands can be used to manage the users:

- `create-user` to create a new user
- `edit-user` to modify an existing user
- `list-users` to list all the existing users

The users are characterized with the following information:

- an identifier,
- a password,
- a flag indicating if the user is enabled,
- a flag indicating if the user has been granted administrator rights (cf. user manual).

The identifier is used for two purposes: logging into the application and building the user's URI. Hence, each user has a distinct URI that will be used to identify who created a notice. Please be aware that this piece of information is published and therefore **the user's identifier should be opaque if you don't want the user to be easily identified from its URI**.

It is impossible to delete an existing user (in order to keep a record of who has used the application and who has created notices) but users can be disabled. A disabled user cannot log in the application.

Creation of a new user

The following command starts the creation of a new user:

```
python3 -m eli_annotation create-user
```

The first piece of information to be entered is the user's identifier. This identifier will be used for logging in the Web application and for building the user's URI. Please be aware that the user's URI will be published and therefore the user's identifier must be opaque.

The full name of the user is entered (that will never be published) and then the password. A confirmation of the password is asked.

Finally, the tool asks if the user has administrator rights (see user manual to understand the differences between a regular user and an administrator).

A new user is always enabled.

The following example shows the creation of a user with administrator rights:

```
python3 -m eli_annotation create-user

User ID: admin
Full name: Administrator
Password:
Password (confirm):
Grant admin privileges [y/N]: Y
```

```
User URI will be: http://data.legi-org.eu/users/admin
User admin has been created.
```

And this example shows the creation of a user without administrator rights:

```
python3 -m eli_annotation create-user

User ID: usr784
Full name: Robert Jones
Password:
Password (confirm):
Grant admin privileges [y/N]: N
User URI will be: http://data.legi-org.eu/users/usr784
User usr784 has been created.
```

Please note that the tool always writes the URI of the user.

Edition of an existing user

The following command starts the edition of the user whose identifier is <user-id>

```
python3 -m eli_annotation edit-user <user-id>
```

For each property, the tool displays the current value and allows to give a new value. The current value is displayed between square brackets. Not giving any value keeps the existing value.

Firstly, it is possible to edit the full name and then the password. The tool asks if the user should be given the administrator rights and finally asks if the user should be enabled. A disabled user still exists but cannot log in the application.

The following example shows the edition of a user. This user is disabled through this sequence; the others properties are not changed.

```
python3 -m eli_annotation edit-user usr784

Full name [Robert Jones]:
Password (leave blank to keep old password):
Currently, user has not been granted admin privileges.
Grant admin privileges [y/N]:
Currently, user is enabled.
Enable user [Y/n]: N
User URI is: http://data.legi-org.eu/users/usr784
User usr784 has been updated.
```

Display of the users list

The following command lists all the existing users:

```
python3 -m eli_annotation list-users
```

The users are listed in CSV format. All the properties but the password are given in the list.

For example:

```
python3 -m eli_annotation list-users

user id,user URI,user name,enabled,admin
admin,http://data.legi-org.eu/users/admin,Administrator,yes,yes
usr784,http://data.legi-org.eu/users/usr784,Robert Jones,no,no
```

If it is possible to use the `--output` option to specify the name of a file where the users list will be written.

Here the users list is written in the `users.json` file

```
python3 -m eli_annotation list-users --output users.json
```

Localization

The localization consists in giving to the application, in a given language, all the texts it can display. There are two kinds of localized resources:

- a JSON file containing the UI messages and the terms from some standard ontologies (e.g. ELI, DC-TERMS, SKOS),
- HTML documents such as the about text or the help on the Excel vocabulary format.

The JSON localization file is used for localizing the Web UI (messages, error names, texts) and for localizing the HTML rendering of the vocabularies (names of the properties from DC-TERMS of SKOS ontologies).

There are operations for:

- getting the JSON localization file for a given language,
- adding (or replacing) the JSON localization file for a given language,
- getting an HTML resource document for a given language,
- adding (or replacing) an HTML resource document for a given language.

It is possible to localize the tool for a new language. The administrator has just to add the JSON localization file and the HTML resource files for this language. One of the languages covered by the JSON files will be used for the UI. Several of these languages will be used for the vocabularies rendering. (see [Configuration](#) (page 7)).

Getting the JSON localization file

The following command gets the JSON localization file for a language:

```
python3 -m eli_annotation get-localization
```

The language is chosen through a dialog. If no localization file exists for this language, the default localization file is returned (in English).

For example:

```
python3 -m eli_annotation get-localization

Enter the language (two-letters code) of the localization file you want to get
```

```
? en

{ "langCode": "en"
  "prefixes": {
    "eli": "http://data.europa.eu/eli/ontology#",
    "rdf": "http://www.w3.org/1999/02/22-rdf-syntax-ns#",
    "dct": "http://purl.org/dc/terms/",
    "skos": "http://www.w3.org/2004/02/skos/core#",
    "euvoc": "http://publications.europa.eu/ontology/euvoc#",
    "elix": "urn:eli-annotation-tool:eli:ontology-extension:",
    "elis": "urn:eli-annotation-tool:skos:ontology-extension:"
  },
  "uiMessages": {
    [...]
  }
}
```

The JSON file is outputted on the standard screen output but it is possible to specify a filename thanks to the `--output` option:

```
python3 -m eli_annotation get-localization --output local.json
```

```
Enter the language (two-letters code) of the localization file you want to get
? en
```

Adding a JSON localization file

The following command adds the `<loc-file.json>` JSON file for a language:

```
python3 -m eli_annotation add-localization <loc-file.json>
```

The language can be specified through a dialog. The default value is read from the JSON file. For example:

```
python3 -m eli_annotation add-localization my-localization.json
```

```
Enter the language (two-letters code) this localization file will be added for
[fr]?
```

Getting an HTML resource file

A single command can be used to get one of the HTML resource file for one language:

```
python3 -m eli_annotation get-resource
```

The type of the resource and the language are chosen through a dialog. If no resource file exists for this language, the default resource file is returned (in English).

For example:

```
python3 -m eli_annotation get-resource
```

```
Choose the resource you want to get
[1] About text of ELI Annotation Tool
[2] Documentation of Excel vocabulary files
```



```
Enter your choice (1 or 2)
? 1

Enter the language (two-letters code) of this resource you want to get
? en

<div>
  <p>The ELI Annotation Tool has...</p>
  [...]
</div>
```

The HTML file is outputted on the standard screen output but it is possible to specify a filename with the `--output` option:

```
python3 -m eli_annotation get-resource --output excel-help.html

Choose the resource you want to get
[1] About text of ELI Annotation Tool
[2] Documentation of Excel vocabulary files

Enter your choice (1 or 2)
? 2

Enter the language (two-letters code) of this resource you want to get
? en
```

Adding an HTML resource file

The following command adds the `<res-file.html>` HTML file as a resource file for one language:

```
python3 -m eli_annotation add-resource <res-file.html>
```

The type of the resource and the language are specified through a dialog. For example:

```
python3 -m eli_annotation add-resource my-excel-help.html

Choose the resource described in this file
[1] About text of ELI Annotation Tool
[2] Documentation of Excel vocabulary files

Enter your choice (1 or 2)
? 2

Enter the language (two-letters code) this resource will be added for
? fr
```

Starting the application

The application is a Web application. To run it, it is necessary to start the server with the following command:

```
python3 -m eli_annotation serve
```

This command has three options:

- `--port` to specify the HTTP port the application listens to (default is 5000)
- `--debug` to start the application in debug mode (all the HTTP requests are logged on the standard screen output)
- `--regenerate` to have the application regenerate the indexes for the existing vocabularies and notices (see user manual) before starting the server. This indexing can be long if there are numerous or big elements.

The application needs an environment variable specifying a secret key that will be used for the users' authentication. This variable is named `SECRET_KEY` and can be set with the command:

```
export SECRET_KEY=MyOwnSecretKey
```

It is also possible to set the variable when starting the server with the shell command:

```
SECRET_KEY=MyOwnSecretKey python3 -m eli_annotation serve
```

Running the application as a daemon

The previous section explains how to run the application. On a production server, it is often preferred to run applications as daemons. Such a thing can be done thanks to the `nohup` command.

```
SECRET_KEY=MyOwnSecretKey nohup python3 -m eli_annotation serve &
```

In this case, the log messages are not written in the console but are saved in a file called `nohup.out` and located in the directory where the `nohup` command has been run.

LICENSE

The ELI Annotation Tool is published under the [European Union Public License \(EURL\)](https://joinup.ec.europa.eu/community/eupl/og_page/eupl) (https://joinup.ec.europa.eu/community/eupl/og_page/eupl).